

## Experiment No. 6

### Aim:

SQL injection vulnerability allow login page to bypass.

### Theory:

#### What is SQL Injection?

SQL injection is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. It generally allows an attacker to view data that they are not normally able to retrieve. This might include data belonging to other users, or any other data that the application itself is able to access. In many cases, an attacker can modify or delete this data, causing persistent changes to the application's content or behaviour.

#### What is the impact of a successful SQL injection attack?

A successful SQL injection attack can result in unauthorized access to sensitive data, such as passwords, credit card details, or personal user information. Many high-profile data breaches in recent years have been the result of SQL injection attacks, leading to reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.

#### SQL Injection examples

There are a wide variety of SQL injection vulnerabilities, attacks, and techniques, which arise in different situations. Some common SQL injection examples include:

- [Retrieving hidden data](#), where you can modify an SQL query to return additional results.
- [Subverting application logic](#), where you can change a query to interfere with the application's logic.
- [UNION attacks](#), where you can retrieve data from different database tables.
- [Examining the database](#), where you can extract information about the version and structure of the database.
- [Blind SQL injection](#), where the results of a query you control are not returned in the application's responses.

#### How to detect SQL injection vulnerabilities?

The majority of SQL injection vulnerabilities can be found quickly and reliably using Burp Suite's [web vulnerability scanner](#).

SQL injection can be detected manually by using a systematic set of tests against every entry point in the application. This typically involves:

- Submitting the single quote character ' and looking for errors or other anomalies.
- Submitting some SQL-specific syntax that evaluates to the base (original) value of the entry point, and to a different value, and looking for systematic differences in the resulting application responses.
- Submitting Boolean conditions such as OR 1=1 and OR 1=2, and looking for differences in the application's responses.
- Submitting payloads designed to trigger time delays when executed within an SQL query, and looking for differences in the time taken to respond.

- Submitting OAST payloads designed to trigger an out-of-band network interaction when executed within an SQL query, and monitoring for any resulting interactions.

## STEP 1-

Here we can see that there is a My Account button in which we will enter the username and password for login as an admin.



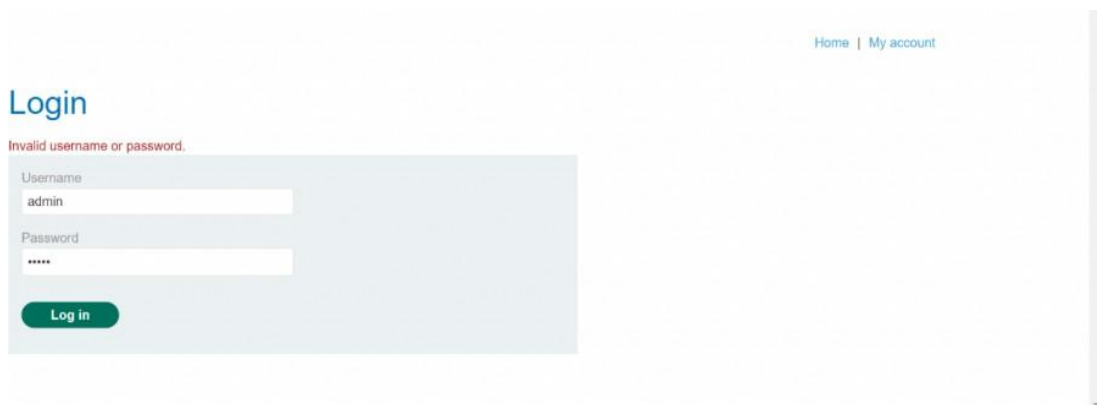
## STEP – 2

After clicking to My Account the screen appears for username and password.



**STEP – 3** After then we will enter username – admin and password – admin and try to get in to website.

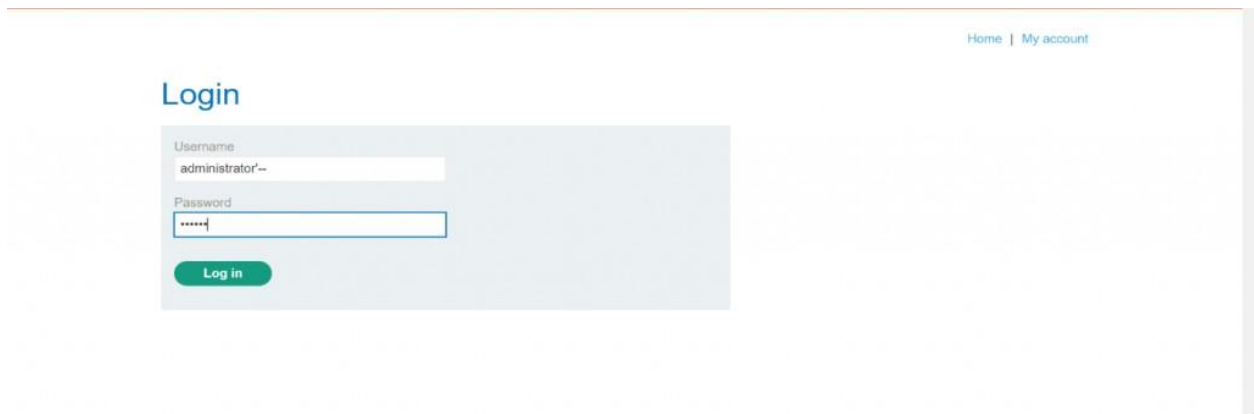
After clicking LOGIN button an error will come and shows as Invalid username and password.



The screenshot shows a web application's login page. At the top right, there are links for "Home" and "My account". The main heading is "Login". Below it, a red error message reads "Invalid username or password.". The login form contains two input fields: "Username" with the value "admin" and "Password" with masked characters "\*\*\*\*\*". A green "Log in" button is positioned below the password field.

#### Step – 4

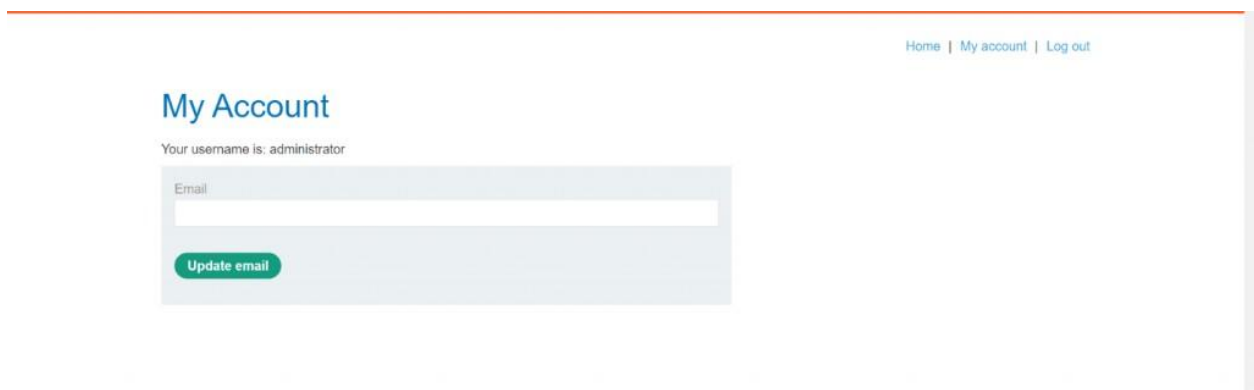
After this we will perform an SQL injection attack that logs in to the application as the administrator user.



The screenshot shows the same login page as before, but the "Username" field now contains the SQL injection payload "administrator'--". The "Password" field remains masked with "\*\*\*\*\*". The green "Log in" button is still present.

#### STEP – 5

After then we can see that we got success in LOGIN as Administrator.



The screenshot shows the "My Account" page. At the top right, there are links for "Home", "My account", and "Log out". The heading is "My Account". Below it, a message states "Your username is: administrator". The form contains an "Email" input field and a green "Update email" button.

## How to prevent SQL injection?

Most instances of SQL injection can be prevented by using parameterized queries (also known as prepared statements) instead of string concatenation within the query.

The following code is vulnerable to SQL injection because the user input is concatenated directly into the query:

```
String query = "SELECT * FROM products WHERE category = '"+ input + "'"; Statement statement = connection.createStatement();
```

```
ResultSet resultSet = statement.executeQuery(query);
```

This code can be easily rewritten in a way that prevents the user input from interfering with the query structure:

```
PreparedStatement statement = connection.prepareStatement("SELECT * FROM products WHERE category = ?");
```

```
statement.setString(1, input);
```

```
ResultSet resultSet = statement.executeQuery();
```

Parameterized queries can be used for any situation where untrusted input appears as data within the query, including the WHERE clause and values in an INSERT or UPDATE statement. They can't be used to handle untrusted input in other parts of the query, such as table or column names, or the ORDER BY clause. Application functionality that places untrusted data into those parts of the query will need to take a different approach, such as white-listing permitted input values, or using different logic to deliver the required behavior.

**Conclusion:** Thus, we have studied SQL injection vulnerability allowing login page to bypass.